
Chapter 10: Storage and File Structure



Chapter 10: Storage and File Structure

- ▶ In the preceding chapters, we have focused on the **high level models of a database**, like the *conceptual or logical level*.
- ▶ In fact, common users should not be burdened unnecessarily with the *physical details of the implementation*, because the goal of a database system is to **simplify and facilitate access to data**.
- ▶ In this chapter, we will discuss the underlying *storage media* and various *data structures* that allow **fast access to data**. In this way, we view a database system at low levels.



Chapter 10: Storage and File Structure

- ▶ **Overview of Physical Storage Media**
- ▶ **Magnetic Disks**
- ▶ **RAID**
- ▶ **Tertiary Storage**
- ▶ **Storage Access**
- ▶ **File Organization**
- ▶ **Organization of Records in Files**
- ▶ **Data-Dictionary Storage**



Classification of Physical Storage Media

- ▶ There are several types of data storage exist in current computer systems – **cache**, **main memory**, **magnetic disk** ...
- ▶ How to classify physical storage media:
 - ▶ *Speed for accessing*
 - ▶ *Cost for per unit of data*
 - ▶ *Reliability*
 - ▶ Data loss on power failure or system crash
 - ▶ Physical failure of the storage device



Physical Storage Media

▶ **Cache**

- ▶ Fastest and most expensive
- ▶ *Volatile* (易失的): data loss on power failure
- ▶ Relatively small
- ▶ Managed by the computer system hardware

▶ **Main memory**

- ▶ Storage for data available to be operated
- ▶ Fast access but generally too small to store the entire database.
- ▶ *Volatile*



Physical Storage Media (cont.)

▶ **Flash memory**

- ▶ Data survives power failure (**non-volatile**)
- ▶ Two types of flash memory
 - ▶ *NAND (Toshiba, 1989)*
 - ▶ *NOR (Intel, 1988)*
- ▶ NAND flash has higher storage capacity for a given cost, and widely used in embedded devices such as digital cameras, phones, and USB keys.



Physical Storage Media (cont.)

▶ **Magnetic-disk**

- ▶ Spinning disk and data read and written magnetically
- ▶ Primary medium for the *long-term storage of data*
- ▶ *Data must be moved from disk to main memory for access, and written back for storage.*
 - ▶ Much slower access than main memory
- ▶ **Direct-access**: read data on disk in any order
- ▶ Data survives power failures and system crashes (*non-volatile*).
 - ▶ Disk failure can destroy data, but rarely happen with a low probability.



Physical Storage Media (cont.)

▶ **Optical storage**

- ▶ *Non-volatile*, data is read optically from a spinning disk using a laser.
- ▶ Reads and writes are slower than with magnetic disk.
- ▶ CD-ROM (640 MB) and DVD (4.7 to 17 GB)
- ▶ Blu-ray disks: 27 GB to 54 GB.
- ▶ Write-one, read-many (WORM) optical disks used for archival storage (CD-R, DVD-R, DVD+R).
- ▶ Multiple write versions also available (CD-RW, DVD-RW, DVD+RW, and DVD-RAM).



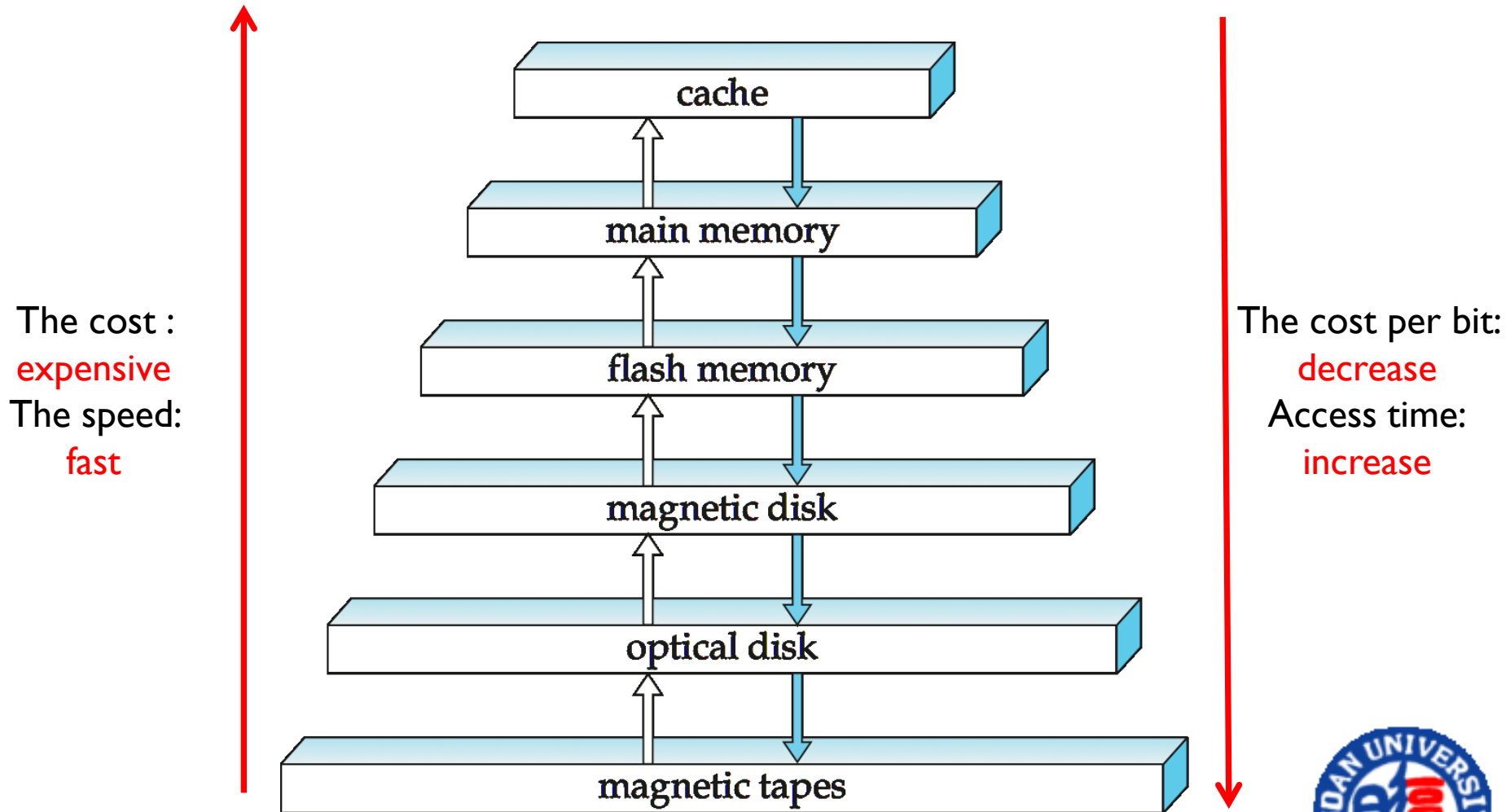
Physical Storage Media (cont.)

▶ **Tape storage**

- ▶ Non-volatile, used primarily for backup (to recover from disk failure), and for archival data.
- ▶ *Sequential-access*: data must be read sequentially from the beginning.
- ▶ Very high capacity (40 to 300 GB tapes available).
- ▶ Tape can be removed from drive, so they are well suited to cheap archival storage.
- ▶ *Tape jukeboxes*(自动光盘机) available for storing massive amounts of data, like data from satellites
 - ▶ Hundreds of TB to even multiple PB.



Storage Hierarchy



Storage Hierarchy (Cont.)

▶ **Primary storage**

- ▶ Fastest media and volatile
- ▶ Example: *cache, main memory*

▶ **Secondary storage**

- ▶ Moderately fast media and non-volatile
- ▶ Example: *flash memory, magnetic disks*

▶ **Tertiary storage**

- ▶ Slowest media and non-volatile
- ▶ Example: *magnetic tape, optical storage*



Magnetic Disks

- ▶ For modern computer system, magnetic disks provide the **most of secondary storage**.
- ▶ The *storage requirements* of large applications have grown faster than the *growths rate of disk capacities*, which means a large database may require hundreds of disks.
- ▶ Flash storage is increasingly becoming a competitor to magnetic disk storage for several applications.



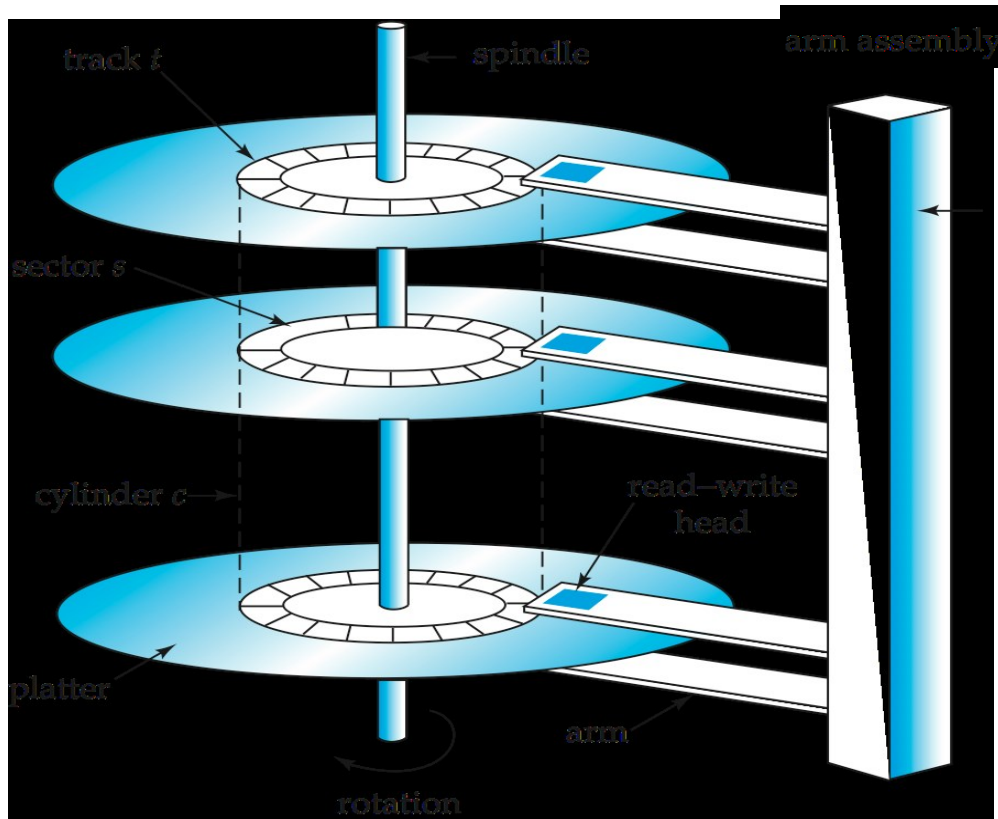
Physical Characteristics of Disks

▶ **Disk platter**

- ▶ a flat, circular shape
- ▶ Its two surfaces are covered with a magnetic material and information is recorded on the surfaces.
- ▶ made from rigid metal or glass.
- ▶ The *disk surface* is logically divided into *tracks*.
- ▶ Tracks are subdivided into *sectors*.
 - ▶ A sector is the *smallest unit of information* that can be read from or written to the disk.



Physical Characteristics of Disks (Cont.)



Currently available disks

Sector sizes: typically 512 bytes

Tracks per platter: about 50,000 to 100,000

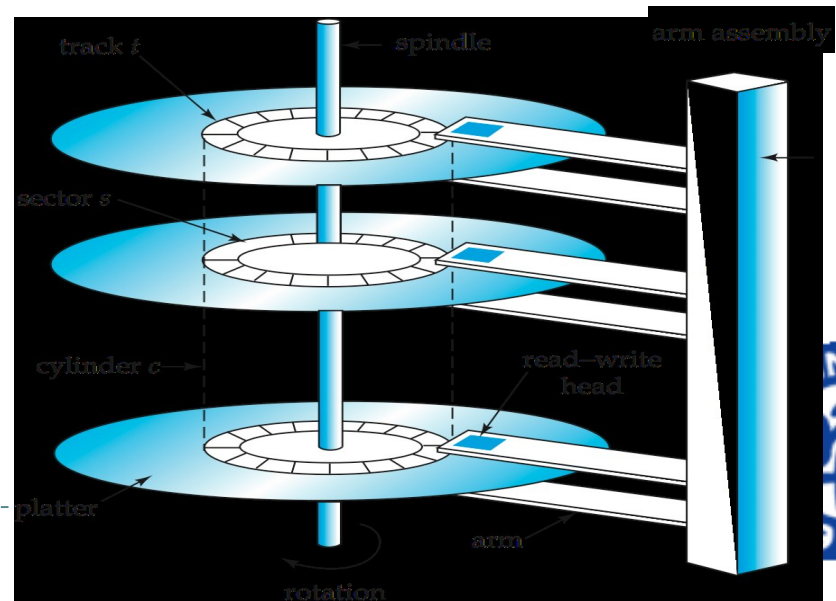
Platters per disk: 1 to 5

- ☞ The inner tracks (closer to the spindle) are of smaller length (around 500 to 1000 sectors).
- ☞ The outer tracks contain more sectors than the inner tracks (around 1000 to 2000 sectors).



Physical Characteristics of Disks (Cont.)

- ▶ **Read–write head**: stores information on a sector magnetically as reversals of the direction of magnetization of the magnetic material.
- ▶ **Disk arm**: the read–write heads of all the tracks are mounted on a disk arm.
- ▶ **Cylinder**: the i^{th} tracks of all the platters together are called the i^{th} cylinder.



Disk Subsystem

- ▶ Multiple disks connected to a computer system through a **controller**:
 - ▶ disk controller is implemented within the disk drive unit.
 - ▶ accepts high-level commands to read or write a sector, and initiates actions.
- ▶ Controllers functionality
 - ▶ *Checksum*
 - ▶ *Bad sector remapping*



Disk Subsystem (Cont.)

- ▶ Disks usually connected directly to computer system.
- ▶ In **Storage Area Networks (SAN)**, a large number of disks are connected by a high-speed network to a number of servers
 - ▶ The disks are usually organized locally using a storage organization technique called *redundant arrays of independent disks* (**RAID**).
- ▶ In **Network Attached Storage (NAS)** networked storage provides a *file system interface* using networked file system protocol, instead of providing a disk system interface.



Performance Measures of Disks

- ▶ The **measures** of the qualities of a disk
 - ▶ *Capacity*
 - ▶ *Access time*
 - ▶ *Data-transfer rate*
 - ▶ *Reliability*



Performance Measures of Disks (Cont.)

- ▶ **Access time** – the time it takes from when a read or write request is issued to when data transfer begins.

It consists of:

- ▶ **Seek time** – *the time it takes to reposition the arm over the correct track.*
 - ▶ Typical seek times range from **2 to 30 milliseconds**, depending on how far the track is from the initial arm position.



Performance Measures of Disks (Cont.)

- ▶ **Access time** – the time it takes from when a read or write request is issued to when data transfer begins.

It consists of:

- ▶ **Seek time** – *the time it takes to reposition the arm over the correct track.*

- ▶ The average seek time is the average of the seek times, measured over a sequence of random requests. (Range between *4 and 10 milliseconds*)
 - If all tracks had the same number of sectors, and we ignore the time to start and stop arm movement, the average seek time is $1/3$ the worst-case seek time.
 - Taking these factors into account, the average seek time is around $1/2$ of the maximum seek time.



Performance Measures of Disks (Cont.)

- ▶ **Access time** – the time it takes from when a read or write request is issued to when data transfer begins.

It consists of:

- ▶ **Rotational latency** – *time it takes for the sector to be accessed to appear under the head*
 - ▶ The average latency time of the disk is one-half the time for a full rotation of the disk.
 - ▶ Rotational speeds of disks range from **4 to 11 milliseconds** per rotation on typical disks (5400 to 15000 rotations per minute).
- ▶ The access time is the *sum of the seek time and the rotational latency* which ranges from **8 to 20 milliseconds**.



Performance Measures of Disks (Cont.)

- ▶ **Data-transfer rate** – *the rate at which data can be retrieved from or stored to the disk*
 - ▶ Current disk systems support maximum transfer rates of **25 to 100 MB per second**.
 - ▶ Transfer rates are significantly lower than the maximum transfer rates for **inner tracks** of the disk.
 - ▶ Multiple disks may share a controller, so the rate that controller can handle is also important
 - ▶ Example : SATA: 150 MB/sec, SATA-II 3Gb (300 MB/sec)
 - ▶ Ultra 320 SCSI: 320 MB/sec, SAS (3 to 6 Gb/sec)
 - ▶ Fiber Channel (FC2Gb or 4Gb): 256 to 512 MB/sec



Performance Measures of Disks (Cont.)

- ▶ **Mean time to failure (MTTF)** – *the average time the disk is expected to run continuously without any failure*
 - ▶ Typically 3 to 5 years
 - ▶ Probability of failure of new disks is quite low and the theoretical MTTF is 500,000 to 1,200,000 hours for a new disk.
 - ▶ An MTTF of 1,200,000 hours for a new disk means that given 1000 relatively new disks, on an average one will fail every 1200 hours.
 - ▶ The claimed MTTF is computed when the disk is new but MTTF will decrease as disk ages.



Disk-block Access

- ▶ A request for disk I/O specifies an address to be accessed which is in the form of a **block number**.
- ▶ **Block**: contiguous sequence of sectors from a single track
 - ▶ Data is transferred between *disk* and *main memory* in blocks.
 - ▶ The term *page* is often used to refer to blocks.
 - ▶ Sizes range from 512 bytes to several kilobytes.
 - ▶ **Smaller blocks**: *more transfers* from disk
 - ▶ **Larger blocks**: more space wasted due to *partially filled blocks*
 - ▶ Typical block sizes today range from 4 to 16 kilobytes.



Disk-block Access (Cont.)

▶ **Disk-block access pattern**

▶ **Sequential access pattern**

- ▶ Successive requests are for successive block numbers, which are on the same track, or on adjacent tracks.
- ▶ To read blocks in sequential access, a disk seek may be required for the first block, but successive requests would either not require a seek.

▶ **Random access pattern**

- ▶ Successive requests are for blocks that are randomly located on disk.
- ▶ Each such request would require a seek.



Optimization of Disk-Block Access

- ▶ Techniques proposed to optimize the disk-block access
 - ▶ **Buffering**
 - ▶ Blocks read from disk are *temporarily stored* in an in-memory buffer to satisfy future request.
 - ▶ **Read-ahead**
 - ▶ *Consecutive blocks from the same track* are read into an in-memory buffer. In sequential access, such technique ensure that many blocks are already in memory when they are requested, minimizing the access time.
 - ▶ **Scheduling**
 - ▶ Algorithms *order pending accesses to tracks* so that disk arm movement is minimized, for example *elevator algorithm*.



Optimization of Disk-Block Access (Cont.)

▶ **File organization**

- ▶ Optimize block access time *by organizing the blocks to correspond to how data will be accessed.*
- ▶ Example: store related information on the same or nearby cylinders, if we expect a file to be accessed sequentially.
- ▶ Older operating systems: IBM mainframe operating system
 - ▶ Provided programmers fine control on placement of files.
 - ▶ Allowing a programmer to reserve a set of cylinders for storing a file.
 - ▶ **Drawback:** this control places a burden on the programmer or system administrator to decide.



Optimization of Disk-Block Access (Cont.)

▶ **File organization**

- ▶ Subsequent operating systems: Unix or Microsoft Windows
 - ▶ Allocate multiple consecutive blocks (an *extent*) at a time to a file, but do not guarantee that all blocks of a file are laid out sequentially.
 - ▶ *Drawbacks*: Files may get *fragmented* over time.
 - If data is inserted to or deleted from the file:
 - Free blocks on disk are scattered, and newly created file has its blocks scattered over the disk
 - Sequential access to a fragmented file results in increased disk arm movement
 - ▶ Some systems have utilities to *defragment* the file system, in order to speed up file access.



Optimization of Disk-Block Access (Cont.)

▶ **Nonvolatile write buffers**

- ▶ Since the contents of main memory are lost in a power failure, information about database updates has to be recorded on disk to survive possible system crashes.
- ▶ For this reason, the performance of update-intensive database applications, such as transaction-processing systems, is *heavily dependent on the speed of disk writes*.
- ▶ *Nonvolatile random-access memory* (NVRAM) to speed up disk writes drastically.



Optimization of Disk-Block Access (Cont.)

▶ **Nonvolatile write buffers**

▶ **Non-volatile RAM:**

- ▶ The contents of NVRAM are not lost in power failure.
- ▶ A common way to implement NVRAM is *to use battery-backed-up RAM*, although flash memory is also increasingly being used for nonvolatile write buffering.

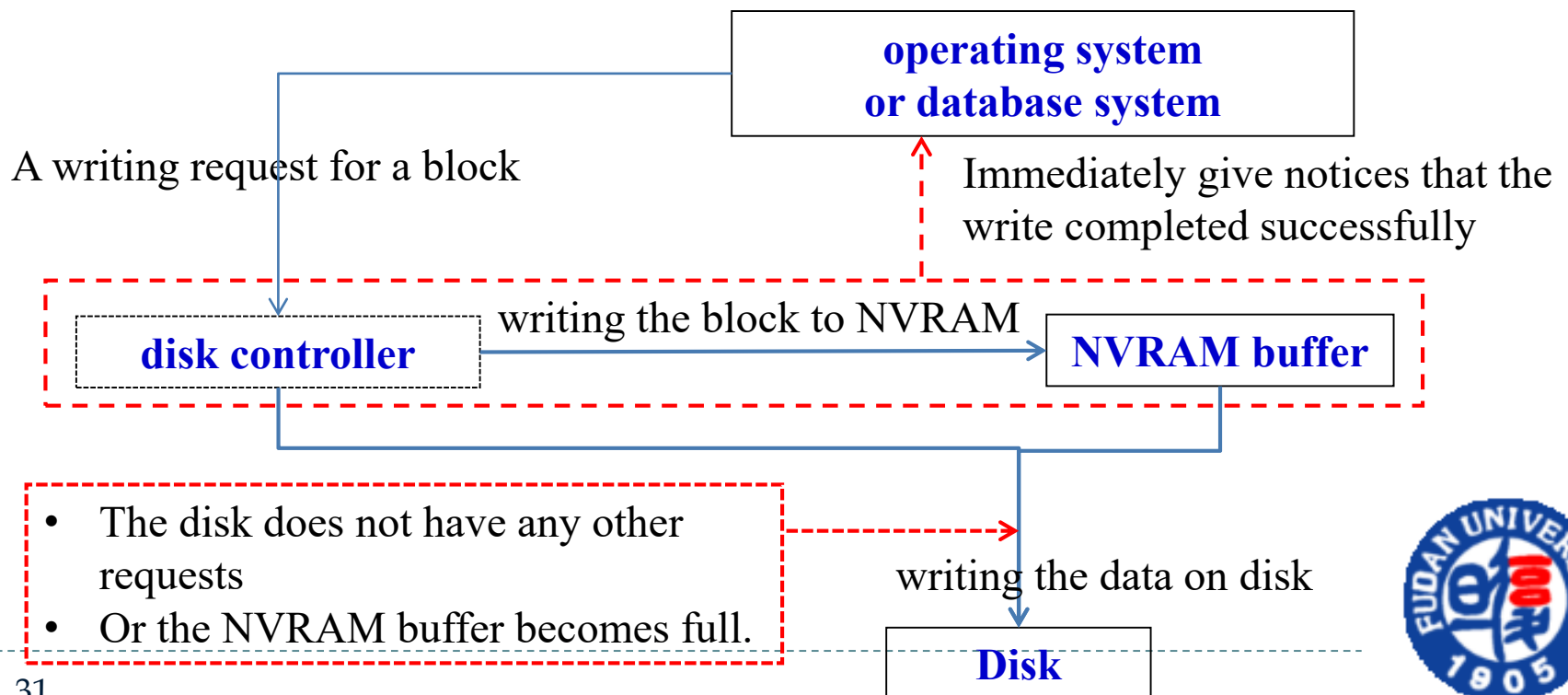


Optimization of Disk-Block Access (Cont.)

► Nonvolatile write buffers

► The Working mechanism

- When the database system requests a block write, it notices a delay only if the NVRAM buffer is full.



Optimization of Disk-Block Access (Cont.)

- ▶ **Log disk**: a disk devoted to writing a sequential log of block updates
 - ▶ Log disks need support from file systems called *journaling file systems* (日志文件系统).
 - ▶ DB system update the sequential log firstly and the log disk can do the write later, and the DB system need not to wait for the write to complete.
 - ▶ All access to the log disk is *sequential*, so the log disk can reorder the writes to *minimize the disk-arm movement*.



Flash Storage

▶ **NOR flash vs NAND flash**

- ▶ NOR flash allows *random access* to **individual words** and has read time comparable to main memory, while reading from NAND flash require an **entire page of data** (512 to 4096 bytes).
- ▶ NAND flash is much cheaper and has higher capacity, which is more widely used.



Flash Storage (Cont.)

▶ **Compared to magnetic disk**

- ▶ *Flash memory can provide much faster random access.*
 - ▶ A page of data can be retrieved in around 1 or 2 microseconds from flash.
 - ▶ a random access on disk would take 5 to 10 microseconds.
- ▶ *Flash memory has a lower transfer rate than magnetic disks.*
 - ▶ Transfer rate around 20 MB/sec
 - ▶ Solid state drives use multiple flash memory chips in parallel, to increase transfer rates to over 200 MB/sec.



Flash Storage (Cont.)

▶ Other Characteristics of flash storage

- ▶ The page of flash memory must be **erased** and **rewritten** subsequently.
- ▶ The erase operation can be performed on a number of pages, called an **erase block** (*takes about 1 to 2 milliseconds*).
 - ▶ Each physical page can be updated only a fixed number of times.
 - ▶ Physical pages have been erased many times are assigned “**cold data**”, while pages that have not been erased many times are used to store “**hot data**”.
 - ▶ This principle of evenly distributing erase operations is called across physical blocks is called *wear leveling*.



Flash Storage (Cont.)

- ▶ Flash memory systems limit the impact of both the slow erase speed and the update limits by *mapping logical page numbers to physical page numbers*.
- ▶ When a logical page is updated, it can be remapped to any already erased physical page, and the original location can be erased later.
- ▶ For quick access, the logical-to-physical page mapping is replicated in an in-memory **translation table**.



RAID

- ▶ The data-storage requirements of some applications have been growing so fast that a large number of disks are needed to store their data.
- ▶ A large number of disks provide opportunities for *improving the accessing rate and the reliability*.
 - ▶ Read or write can be executed *in parallel for speeding*.
 - ▶ Redundant information can be stored on multiple disks for *preventing data loss*.
- ▶ A variety of **disk-organization techniques**, collectively called **RAID (redundant arrays of independent disks)**, have been proposed for *high performance* and *reliability*.



RAID (Cont.)

- ▶ **RAID: *Redundant Arrays of Independent Disks***
 - ▶ **High capacity** and **high speed** by using multiple disks in parallel.
 - ▶ **High reliability** by storing data redundantly, so that data can be recovered even if a disk fails .
 - ▶ “I” in RAID originally stood for “*inexpensive*” but today the “I” is interpreted as “*independent*”.



Improvement of Reliability via Redundancy

- ▶ **Redundancy** – *store extra information that can be used to rebuild information lost in a disk failure*
 - ▶ Example : *mirroring* (or *shadowing*), *parity bits*
- ▶ The simplest (but most expensive) approach to introducing **redundancy** is to duplicate every disk -- *mirroring*.
 - ▶ E.g.: a logical disk consists of two physical disks, and every write is carried out on both disks.
 - ▶ If one of the disks fails, the data can be read from the other.



Improvement in Performance via Parallelism

- ▶ With **multiple disks**, the transfer rate will be improved by *read or write data from multiple disks at the same time*.
- ▶ We can improve transfer rate by *striping data* across multiple disks.
 - ▶ *Bit-level striping*
 - ▶ *Block-level striping*



Improvement in Performance via Parallelism

- ▶ **Bit-level striping** – *split the bits of each byte across multiple disks*
 - ▶ Suppose we have an array of eight disks, just **write bit i of each byte to disk i** .
 - ▶ The array of eight disks can be treated as a single disk with sectors that are eight times the normal size, and, more important, that has *eight times the transfer rate*.



Improvement in Performance via Parallelism

- ▶ **Block-level striping** – *stripes blocks across multiple disks*
 - ▶ E.g.: with n disks, block i of a file goes to **disk $(i \bmod n) + 1$**
 - ▶ When reading a large file, block-level striping *fetches n blocks at a time in parallel from the n disks*, giving a high data-transfer rate for large reads.
 - ▶ When a single block is read, the data-transfer rate is the same as on one disk, but the remaining $n - 1$ disks are free to perform other actions.



Improvement in Performance via Parallelism

- ▶ *Block-level striping* is more commonly used than *bit-level striping* and other levels of striping.
- ▶ There are **two main goals of parallelism** in a disk system:
 - ▶ *Load balance multiple small accesses* to increase throughput.
 - ▶ *Parallelize large accesses* to reduce response time.



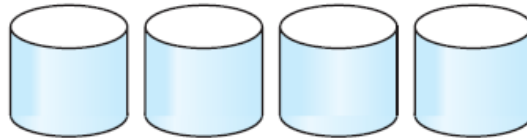
RAID Levels

- ▶ **Mirroring** provides *high reliability*, but it is *expensive*.
- ▶ **Striping** provides *high data-transfer rates*, but does not improve *reliability*.
- ▶ Various alternative schemes aim to provide *redundancy* at *lower cost* by combining *disk striping* with “*parity*” *bits*.
- ▶ The schemes are classified into **RAID levels**.
- ▶ RAID organizations: *Cost, Performance, Reliability*



RAID Levels (Cont.)

- ▶ **RAID Level 0: Block striping without redundancy**
 - ▶ Used in *high-performance applications* where data loss is not critical.

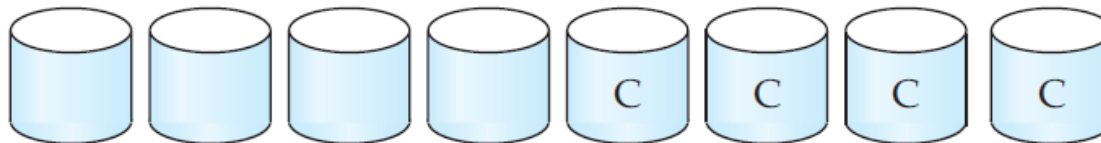


(a) RAID 0: nonredundant striping



RAID Levels (Cont.)

- ▶ **RAID Level 1:** *Mirrored disks with block striping*
 - ▶ Offer best write performance but cost more disks.
 - ▶ Some vendors: use the term **RAID level 1+0** or **RAID level 10** to refer to mirroring with striping; use the term **RAID level 1** to refer to mirroring without striping.
 - ▶ RAID level 1 is popular for applications such as *storing log files in a database system*, since it offers the **best write performance**.



(b) RAID 1: mirrored disks



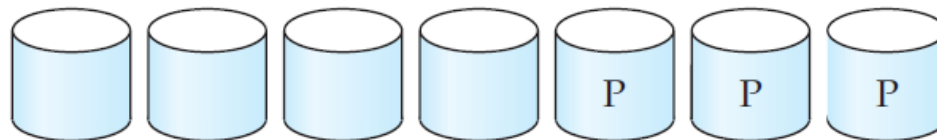
RAID Levels (Cont.)

- ▶ **RAID Level 2: *Memory-Style Error-Correcting-Codes (ECC) with bit striping***
 - ▶ Memory system use **parity bits** for error detection and correction.
 - ▶ Each byte in a memory system may have a parity bit. It records whether the numbers of bits in the byte that are set to 1 is **even** (**parity = 0**) or **odd** (**parity = 1**).
 - ▶ All **1-bit errors** will be detected by the memory system.
 - ▶ **Error-correcting schemes** store 2 or more extra bits, and can reconstruct the data if a single bit gets damaged.



RAID Levels (Cont.)

- ▶ **RAID Level 2: *Memory-Style Error-Correcting-Codes (ECC) with bit striping***
- ▶ **Error-correcting codes** can be used directly in disk arrays by striping bytes across disks.
 - ▶ E.g., for each byte, disk0 stores the first bit, disk1 stores the second bit, ... disk7 stores the eighth bit. And the *error-correction bits are stored in further disks*.
 - ▶ If one of the disks fails, the remaining bits of the byte and the associated error-correction bits can be read from other disks, and can be used to reconstruct the damaged data.



(c) RAID 2: memory-style error-correcting codes



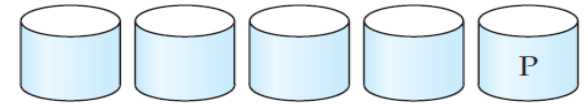
RAID Levels (Cont.)

▶ **RAID Level 3: *Bit-Interleaved Parity***

- ▶ *A single parity bit* is enough for error correction, not just detection, since we know which disk has failed.
 - ▶ When writing data, corresponding parity bits must be computed and written to a parity bit disk.
 - ▶ The *idea of detection and error correction*:
 - The system knows *exactly the damaged sector*.
 - For each bit in the sector, the system can figure out whether it is a 1 or a 0 by computing the parity of the corresponding bits from sectors in the other disks.
 - If the parity of the remaining bits is equal to the stored parity, the missing bit is 0; otherwise, it is 1.



RAID Levels (Cont.)



(d) RAID 3: bit-interleaved parity

▶ **RAID Level 3: Bit-Interleaved Parity**

▶ **RAID level 3 VS RAID level 2, level 1**

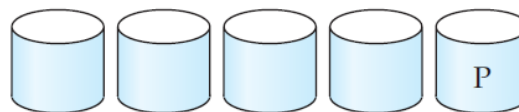
- ▶ RAID level 3 is *as good as* RAID level 2, but is *less expensive* in the number of extra disks. (RAID level 2 is not used in practice.)
- ▶ RAID level 3 has **two benefits** over RAID level 1:
 - Needs only *one parity disk* for several regular disks
 - *Faster transfer rate for a block* than level 1: reads and writes of a byte are spread out over multiple disks, with *N-way striping of data*, the transfer rate for reading or writing a single block is N times faster than level 1 organization using *block striping*.
- ▶ But, RAID level 3 supports *fewer I/O* operations per second since every disk has to participate in every I/O request.



RAID Levels (Cont.)

▶ **RAID Level 4:** *Block-Interleaved Parity*

- ▶ Uses *block-level striping*, and *keeps a parity block* on a separate disk for corresponding blocks from N other disks.
 - ▶ When writing data block, corresponding block of parity bits must also be computed and written to a parity disk.
 - ▶ If one of the disks fails, the parity block can be used with the corresponding blocks from the other disks to restore the blocks of the failed disk.



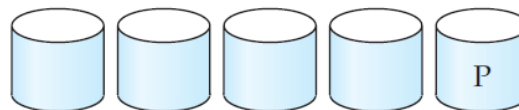
(e) RAID 4: block-interleaved parity



RAID Levels (Cont.)

▶ **RAID Level 4:** *Block-Interleaved Parity*

- ▶ *A block read accesses only one disk*, allowing other requests to be processed by the other disks.
- ▶ Thus, the data-transfer rate for each access is slower, but multiple read accesses can proceed in parallel, leading to *a higher overall I/O rate*.
- ▶ The transfer rates for large reads is high, since all the disks can be read in parallel.
- ▶ large writes also have high transfer rates, since the data and parity can be written in parallel.



(e) RAID 4: block-interleaved parity



RAID Levels (Cont.)

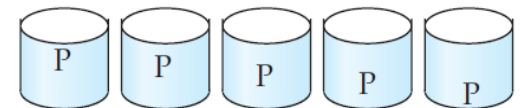
► **RAID Level 5:** *Block-Interleaved Distributed Parity*

- *Partitions data and parity among all $N + 1$ disks*, rather than storing data in N disks and parity in 1 disk.

With an array of 5 disks:

1. The parity block, labeled P_k , for logical blocks $4k, 4k + 1, 4k + 2, 4k + 3$.
2. P_k is stored in disk $k \bmod 5$.
3. The corresponding blocks of the other four disks store the 4 data blocks $4k$ to $4k+3$.

P0	0	1	2	3
4	P1	5	6	7
8	9	P2	10	11
12	13	14	P3	15
16	17	18	19	P4



(f) RAID 5: block-interleaved distributed parity



RAID Levels (Cont.)

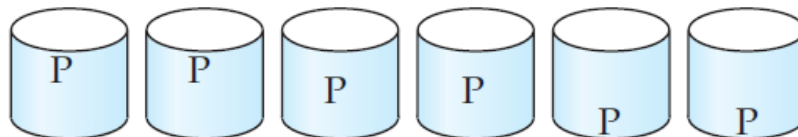
- ▶ **RAID Level 5:** *Block-Interleaved Distributed Parity*
 - ▶ Compared with Level 4, Level 5 *increases the total number of requests* that can be met in a given amount of time.
 - ▶ All disks can participate in satisfying read requests.
 - ▶ *Level 5 subsumes Level 4:* provides same benefits, but *avoids bottleneck of parity disk.*
 - ▶ **Note:** *A parity block cannot store parity for blocks in the same disk*, since a disk failure would result in loss of data as well as of parity, and hence would not be recoverable.
 - ▶ So level 4 is not used in practice.



RAID Levels (Cont.)

▶ **RAID Level 6: $P+Q$ Redundancy scheme**

- ▶ Similar to Level 5, but stores extra redundant information to guard against *multiple disk failures*.
- ▶ Instead of using parity, level 6 uses *error-correcting codes*.
- ▶ Better reliability than Level 5 at a higher cost
 - ▶ In level 6, 2 bits of redundant data are stored for every 4 bits of data — unlike 1 parity bit in level 5.
- ▶ The example system can **tolerate two disk failures**. It is not used widely.



(g) RAID 6: $P + Q$ redundancy



Choice of RAID Level

- ▶ Factors in choosing RAID level
 - ▶ *Monetary cost* of extra disk-storage requirements
 - ▶ *Performance* in terms of number of *I/O operations per second*
 - ▶ *Performance when a disk has failed*
 - ▶ *Performance during rebuild*



Choice of RAID Level (Cont.)

- ▶ **RAID 0** is used only when data safety is not important or data can be recovered quickly from other sources.
- ▶ **RAID 2 and 4** never used since they are subsumed by 3 and 5.
- ▶ **Bit striping (level 3)** is inferior to **block striping (level 5)**.
 - ▶ Block striping gives as good data-transfer rates for large transfers, while using *fewer disks for small transfers*.
 - ▶ For small transfers, the disk access time dominates anyway, so the benefit of parallel reads diminishes.
 - ▶ In fact, level 3 may perform worse than level 5 for a small transfer.
- ▶ **RAID 6** is rarely used since RAID 1 and 5 offer adequate safety for most applications.



Choice of RAID Level (Cont.)

- ▶ In many cases, people have to make choice between **RAID 1** and **RAID 5**.
 - ▶ RAID 1 provides *much better write performance* than RAID 5.
 - ▶ RAID 5 had *lower storage cost* than RAID 1.
 - ▶ For applications where data are *read frequently*, and *written rarely*, level 5 is the preferred choice.



Hardware Issues

- ▶ **Software RAID:** RAID can be implemented with no change at the hardware level, using only software modification.
- ▶ **Hardware RAID:** systems with special hardware support
 - ▶ There are significant benefits to be had by building special-purpose hardware to support RAID.
 - ▶ Hardware RAID implementations can use *nonvolatile RAM* to record writes before they are performed.

Hardware Issues (Cont.)

▶ **Hardware RAID**

▶ In case of **power failure**

- ▶ When the system recovering, it retrieves information about any incomplete writes from nonvolatile RAM and then completes the writes.
- ▶ Without such hardware support, extra work needs to be done to detect blocks that may have been partially written before power failure.



Tertiary Storage

▶ Optical Disks

- ▶ Popular medium for distributing software, multimedia data such as audio and images, and other electronically published information.
- ▶ Storage capacity ranged from 600MB to 54GB.
- ▶ Slower than magnetic disk.

▶ Magnetic Tapes

- ▶ Magnetic tapes are relatively permanent, and can hold large volumes of data.
- ▶ Very slowly access time limited by sequential access.
- ▶ Be Used mainly for backup, for storage of infrequently used information, and as an off-line medium for transferring information from one system to another.



File Organization

- ▶ A **database** is mapped into *a number of different files* that are maintained by the underlying operating system.
- ▶ A **file** is organized logically *as a sequence of records*.
- ▶ These **records** are mapped onto *disk blocks*.
- ▶ Each **file** is logically partitioned into *fixed-length storage units* called *blocks* – the units of both storage allocation and data transfer.
- ▶ There are several assumptions before we deal with database in terms of file:
 - ▶ *A block can contain many records.*
 - ▶ *No record is larger than a block.*
 - ▶ *Each record is entirely contained in a single block.*



File Organization (Cont.)

- ▶ Approaches to **mapping the database to files**
 - ▶ *Use several files, and to store records of only one **fixed length** in any given file.*
 - ▶ *Structure files to be accommodated to multiple lengths for records.*
- ▶ Many of the techniques used for the former can be applied to the **variable-length** case.



Fixed-Length Records

```
type instructor = record
    ID varchar (5);
    name varchar(20);
    dept_name varchar (20);
    salary numeric (8,2);
end
```

- ▶ Suppose the *instructor* record is 53 bytes long as listed above. The simplest approach is to use the first 53 bytes to store the first record and the next 53 for the second and so on.
- ▶ **Two problems** with this simple approach:
 - ▶ Unless the block size happens to be a multiple of 53 (which is unlikely), some records will *cross block boundaries*.
 - ▶ **Modification**: do not allow records to cross block boundaries and ignore the remaining bytes at the end.



Fixed-Length Records

- ▶ **Two problems** with this simple approach:
 - ▶ It is *difficult to delete a record* from this structure.
 - ▶ Considering the *deletion* of the fixed-length records. Suppose there is a table as below, we will try to delete record i and reorganize file in three ways:
 - ▶ Delete record i , then

- *move records $i + 1, \dots, n$ to $i, \dots, n - 1$*
- *move record n to i*
- *do not move records, but link all free records on a free list*

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Fixed-Length Records

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

*move records $i + 1, \dots, n$
to $i, \dots, n - 1$*

move record n to i

record 0	10101	Srinivasan	Comp. Sci.	65000
record 1	12121	Wu	Finance	90000
record 2	15151	Mozart	Music	40000
record 11	98345	Kim	Elec. Eng.	80000
record 4	32343	El Said	History	60000
record 5	33456	Gold	Physics	87000
record 6	45565	Katz	Comp. Sci.	75000
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000



Deletion Issues

- ▶ It is undesirable to move records to occupy the space freed by a deleted record, since doing so *requires additional block accesses*.
- ▶ Since *insertions tend to be more frequent than deletions*, it is acceptable to *leave open* the space occupied by the deleted record, and to wait for a subsequent insertion before reusing the space.
- ▶ To make it *easy to find available space*, people have developed additional structures, like **free list**.



Free Lists

- ▶ Allocate a certain number of bytes as **file header** at the *beginning of the file*, which will contain a variety of information about the file.
- ▶ Store the *address of the first deleted record* in the file header.
- ▶ Use this first record to store the address of the second deleted record, and so on...
- ▶ Intuitively, we can think of these stored addresses as *pointers*, since they point to the location of a record.



Free Lists (Cont.)

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

Insertion Issues with Free List

- On insertion of a new record, we use the record pointed to by the header.
- Change the header pointer to the next available record.
- If no space available, add the new one to the end of the file.

Record 1, 4 and 6 was deleted. The address of record 1 is stored in header and the address of record 4 is stored in record 1 and so on.



Variable-Length Records

- ▶ **Variable-length records** arise in database systems
 - ▶ Storage of *multiple record types* in a file.
 - ▶ Record types that allow *variable lengths* for one or more fields such as strings (*varchar*).
 - ▶ Record types that allow *repeating fields*, such as arrays or multisets.
- ▶ Two issues about variable-length records
 - ▶ *How to represent a single record in such a way that individual attributes can be extracted easily?*
 - ▶ *How to store variable-length records within a block, such that records in a block can be extracted easily?*

Variable-Length Record (Cont.)

- ▶ A **variable length record** consists of *fixed-length attributes* and *variable-length attributes*.
 - ▶ **Fixed-length attributes**, such as numeric values, dates, or fixed-length character strings are allocated as many bytes as required to store their value.
 - ▶ **Variable-length attributes**, such as *varchar* types, are represented in the initial part of the record by a pair (*offset, length*).
 - ▶ **Offset** denotes *the beginning bits*
 - ▶ **Length** denotes *size of the variable-length attributes*
 - ▶ The values for these attributes are stored consecutively, after the initial fixed-length part of the record.



Variable-Length Record (Cont.)

- ▶ The **initial part of the record** stores a fixed size of *information about each attribute*, whether it is fixed-length or variable-length.
- ▶ **Null-value bitmap** can indicate which attributes have *null* value.

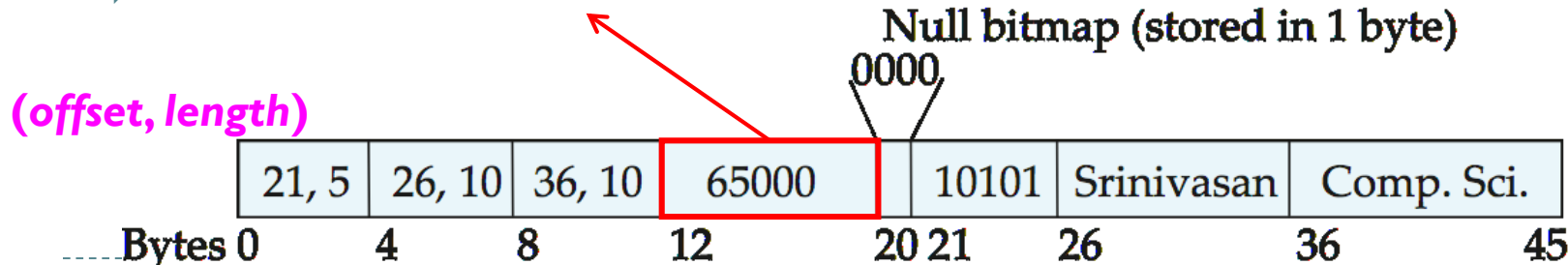
▶ E.g.: *instructor*

- ▶ *ID, name, and dept_name*:
variable-length strings

- ▶ *salary*: a fixed-sized number

type instructor = record

ID varchar (5);
name varchar(20);
dept_name varchar (20);
salary numeric (8,2);
end



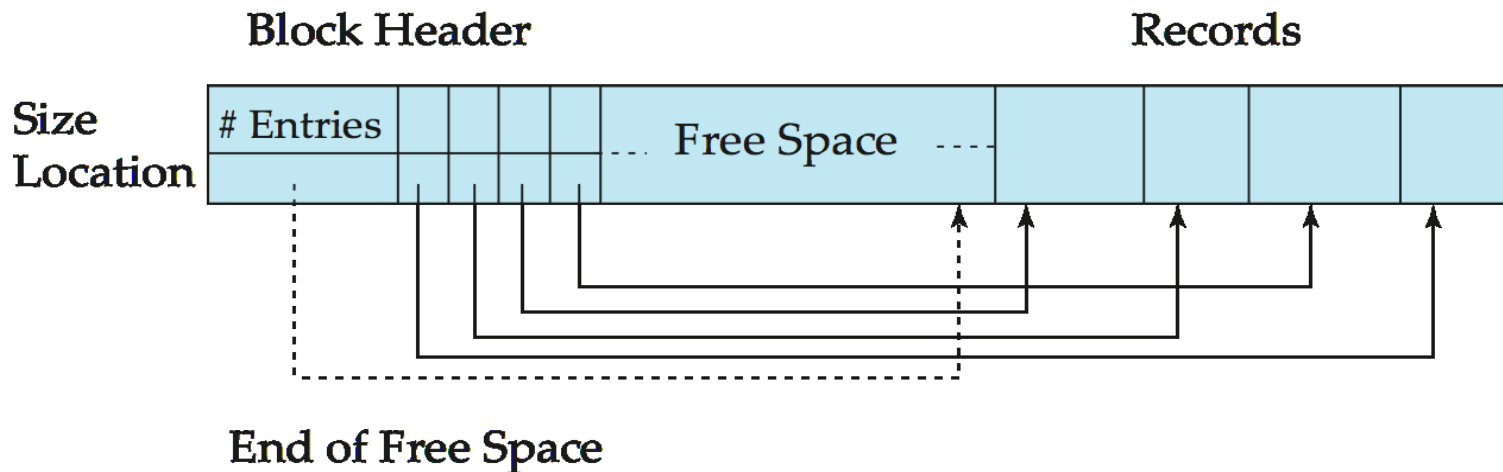
Slotted-Page Structure

- ▶ There is a next problem we need to address:
 - ▶ *How do we store variable-length record in a block?*
- ▶ The **slotted-page structure** is commonly used for organizing records within a block.
- ▶ There is a **header** at the beginning of each block, containing the following information:
 - ▶ The *number of record entries* in the header.
 - ▶ The *end of free space* in the block.
 - ▶ An *array* whose entries contain the location and size of each record.



Slotted-Page Structure (Cont.)

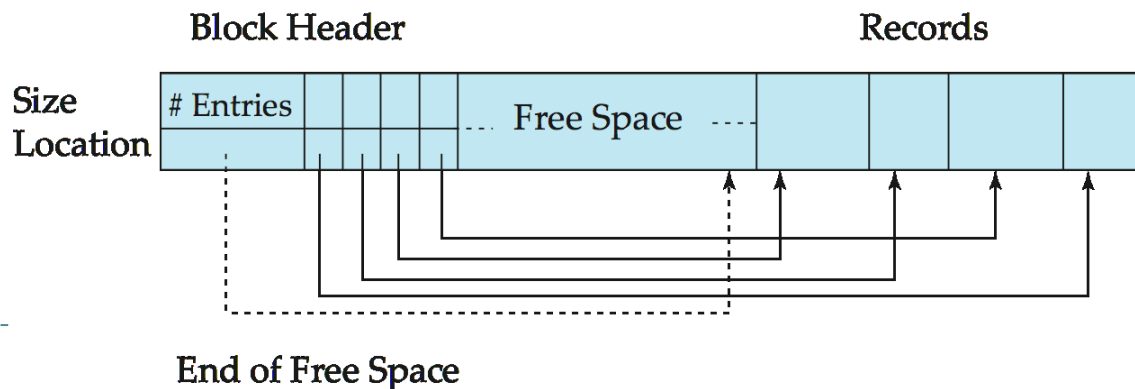
- ▶ The *actual records* are allocated *contiguously* in the block, starting from the end of the block.
- ▶ The *free space* in the block is *contiguous*, between the final entry in the header array, and the first record.



Slotted-Page Structure (Cont.)

► Inserting or Deleting a record

- **Insert**: space is allocated for it at *the end of free space*, and an entry containing its size and location is added to the *header*.
- **Delete**: the space that it occupies is freed, and its *entry* is set to *deleted* (its size is set to -1 , for example).
 - The records in the block before the deleted record can be moved, so that the free space created by the deletion gets occupied, and all free space is again between the final entry in the header array and the first record.



Handle Large Data Items

- ▶ Databases often store data that can be much larger than a disk block.
 - ▶ For instance, an image or an audio recording may be multiple megabytes in size, while a video object may be multiple gigabytes in size.
- ▶ Recall that SQL supports the types **blob** and **clob**, which store binary and character large objects.
- ▶ Most relational databases restrict the size of a record to be no larger than the size of a block to *simplify buffer management and free-space management*.



Handle Large Data Items (Cont.)

- ▶ Large objects are often stored in a *special file* (or collection of files) instead of being stored with the other (short) attributes of records in which they occur.
- ▶ A *(logical) pointer* to the object is then stored in the record containing the large object.
- ▶ Large objects are often represented using **B+-tree file** organizations.



Organization of Records in Files

- ▶ Up to this point, we have talked a lot about *how records are represented in a file structure*.
- ▶ A relation is a set of records. Given a set of records, the next question is *how to organize the records in a file*.
- ▶ Several of the possible ways of organizing records in files are
 - ▶ *Heap file organization*
 - ▶ *Sequential file organization*
 - ▶ *Hash file organization*
 - ▶ *Multitable clustering file organization*



Organization of Records in Files (Cont.)

- ▶ **Heap file organization** – *any record can be placed anywhere in the file where there is space.*
 - ▶ There is no ordering of records.
 - ▶ Typically, there is a single file for each relation.
- ▶ **Sequential file organization** – *records are stored in sequential order, according to the value of a “search key” of each record.*



Organization of Records in Files (Cont.)

- ▶ **Hashing file organization** – *a hash function computed on some attribute of each record; the result specifies in which block of the file the record should be placed.*
- ▶ Records of each relation may be stored in a separate file. However, in a **multi-table clustering file organization**, *records of several different relations can be stored in the same file.*
- ▶ Further, *related records* of the different relations are stored on the same block, so that one I/O operation fetches related records from all the relations.



Sequential File Organization

- ▶ A **sequential file** is designed for *efficient processing of records in sorted order based on some search key*.
- ▶ A **search-key** is any attribute or set of attributes. It need not be the primary key, or even a superkey.
- ▶ To permit fast retrieval of records in search-key order, we *chain together records by pointers* (the pointer in each record points to the next record in search-key order)
- ▶ To minimize the number of block accesses in sequential file processing, we *store records physically in search-key order*, or as close to search-key order as possible.



Sequential File Organization (Cont.)

- ▶ A sequential file of *instructor* records
- ▶ the records are stored in search-key order, using *ID* as the search key.
- ▶ All records are *linked by pointer chain*.

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	



Drawbacks

- It is difficult to *maintain physical sequential order* as records are inserted and deleted.
- Since, it is costly to move many records as a result of a single insertion or deletion.

Sequential File Organization (Cont.)

▶ **Deletion**

- ▶ Manage deletion by using pointer chains

▶ **Insertion**

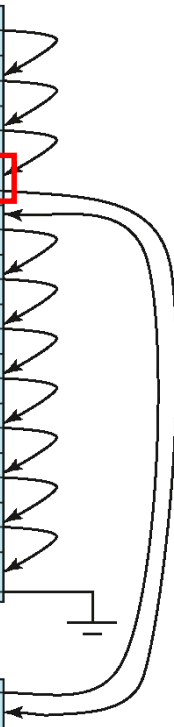
- ▶ Locate the record in the file that comes *before* the record to be inserted in search-key order.
- ▶ If there is *a free record* within the same block as this record, insert the new record there.
- ▶ If no free space, insert the record in an *overflow block*.
- ▶ In either case, *adjust the pointers* so as to chain together the records in search-key order.



Sequential File Organization (Cont.)

10101	Srinivasan	Comp. Sci.	65000	
12121	Wu	Finance	90000	
15151	Mozart	Music	40000	
22222	Einstein	Physics	95000	
32343	El Said	History	60000	
33456	Gold	Physics	87000	
45565	Katz	Comp. Sci.	75000	
58583	Califieri	History	62000	
76543	Singh	Finance	80000	
76766	Crick	Biology	72000	
83821	Brandt	Comp. Sci.	92000	
98345	Kim	Elec. Eng.	80000	

32222	Verdi	Music	48000	
-------	-------	-------	-------	--



The structure allows fast insertion of new records, but forces sequential file-processing applications to process records in an order that **does not match the physical order of the records.**

Insert a record: (32222, Verdi, Music, 48000).

The inserted record is stored in an **overflow block** and the pointer chain is updated to keep the file in **search-key** order



Sequential File Organization (Cont.)

- ▶ If relatively few records need to be stored in overflow blocks, this approach works well.
- ▶ However, the *correspondence between search-key order and physical order* may be totally lost over a period of time because of the deletion and insertion.
- ▶ At this point, the file should be **reorganized** so that it is once again physically in sequential order.
- ▶ Such reorganizations are costly, and must be done during times when the system load is low.



Multitable Clustering File Organization

- ▶ Many relational database systems *store each relation in a separate file*, so that they can take full advantage of the file system that the operating system provides.
- ▶ Usually, tuples of a relation can be represented as fixed-length records. Thus, *relations can be mapped to a simple file structure*.
- ▶ This simple implementation of a relational database system is well *suited to low-cost database implementations as in*, for example, embedded systems or portable devices.
- ▶ In such systems, the size of the database is small, so little is gained from a sophisticated file structure.



Multitable Clustering File Organization

- ▶ **However**, many large-scale database systems do not rely directly on the underlying operating system for file management.
- ▶ Instead, one large operating system file is allocated to the database system. The database system stores all relations in this one file, and manages the file itself.
- ▶ Even if multiple relations are stored in a single file, by default most databases *store records of only one relation in a given block*. This simplifies data management.
- ▶ **However**, in some cases it *can be useful to store records of more than one relation in a single block*.



Multitable Clustering File Organization

- ▶ Suppose there are two relations as below

department

<i>dept_name</i>	<i>building</i>	<i>budget</i>
Comp. Sci.	Taylor	100000
Physics	Watson	70000

instructor

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp. Sci.	65000
33456	Gold	Physics	87000
45565	Katz	Comp. Sci.	75000
83821	Brandt	Comp. Sci.	92000



Multitable Clustering File Organization

- ▶ SQL query for the university database:

```
select dept_name, building, budget, ID, name, salary  
from department natural join instructor;
```

- ▶ For each tuple of *department*, the system must locate the *instructor* tuples with the same value for *dept_name*.
- ▶ In the worst case, each record will reside on a different block, forcing us to *do one block read for each record required by the query*.
- ▶ So, it can design a file structure for *efficient execution of queries involving the natural join* of *department* and *instructor*.



Multitable Clustering File Organization

- ▶ The *instructor* tuples for each *ID* are stored near the *department* tuple for the corresponding *dept_name*.

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000

- ▶ The *dept_name* attribute is omitted from *instructor* records since it can be inferred from the associated *department* record.
- ▶ This structure mixes together tuples of two relations, but allows for *efficient processing of the join*.



Multitable Clustering File Organization

- ▶ This **multitable clustering file organization** is a file organization, that *stores related records of two or more relations in each block*.
- ▶ Such a file organization allows us to read records that would satisfy the join condition by using one block read.
- ▶ Thus, we are able to process this particular query more efficiently.

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000



Multitable Clustering File Organization

- ▶ **But** it results in slowing processing of other types of queries.
- ▶ E.g.:

select *
from *department*;
- ▶ This query requires more block accesses than it did in the scheme under which we stored each relation in a separate file.
- ▶ Since each block now contains significantly fewer *department* records.

Comp. Sci.	Taylor	100000
45564	Katz	75000
10101	Srinivasan	65000
83821	Brandt	92000
Physics	Watson	70000
33456	Gold	87000

Multitable Clustering File Organization

- ▶ To efficiently locate all tuples of *department*, we can *chain together all the records of that relation using pointers*.

Comp. Sci.	Taylor	100000	
45564	Katz	75000	
10101	Srinivasan	65000	
83821	Brandt	92000	
Physics	Watson	70000	
33456	Gold	87000	



Data Dictionary Storage

- ▶ A relational database system needs to *maintain data about the relations*, such as the schema of the relations.
- ▶ In general, such “data about data” is referred to as *metadata*.
- ▶ Relational schemas and other metadata about relations are stored in a structure called the **data dictionary** or **system catalog**.



Data Dictionary Storage

- ▶ Among the types of information that the system must store are these:
 - ▶ *Names of relations.*
 - ▶ *Names of the attributes of each relation.*
 - ▶ *Domains and lengths of attributes.*
 - ▶ *Names of views defined on the database, and definitions of those views.*
 - ▶ *Integrity constraints* (for example, key constraints).



Data Dictionary Storage (Cont.)

- ▶ Many systems keep the following data on *users of the system*:
 - ▶ *Names of authorized users.*
 - ▶ *Authorization and accounting information about users.*
 - ▶ *Passwords or other information used to authenticate users.*
- ▶ The database may store *statistical and descriptive data about the relations*:
 - ▶ *Number of tuples in each relation.*
 - ▶ *Method of storage for each relation* (for example, clustered or nonclustered).



Data Dictionary Storage (Cont.)

- ▶ The data dictionary may also note the *storage organization* (sequential, hash, or heap) of relations, and the *location* where each relation is stored.
- ▶ Store information about *each index on each of the relations*:
 - ▶ *Name of the index.*
 - ▶ *Name of the relation being indexed.*
 - ▶ *Attributes on which the index is defined.*
 - ▶ *Type of index formed.*



Relational Representation of System Metadata

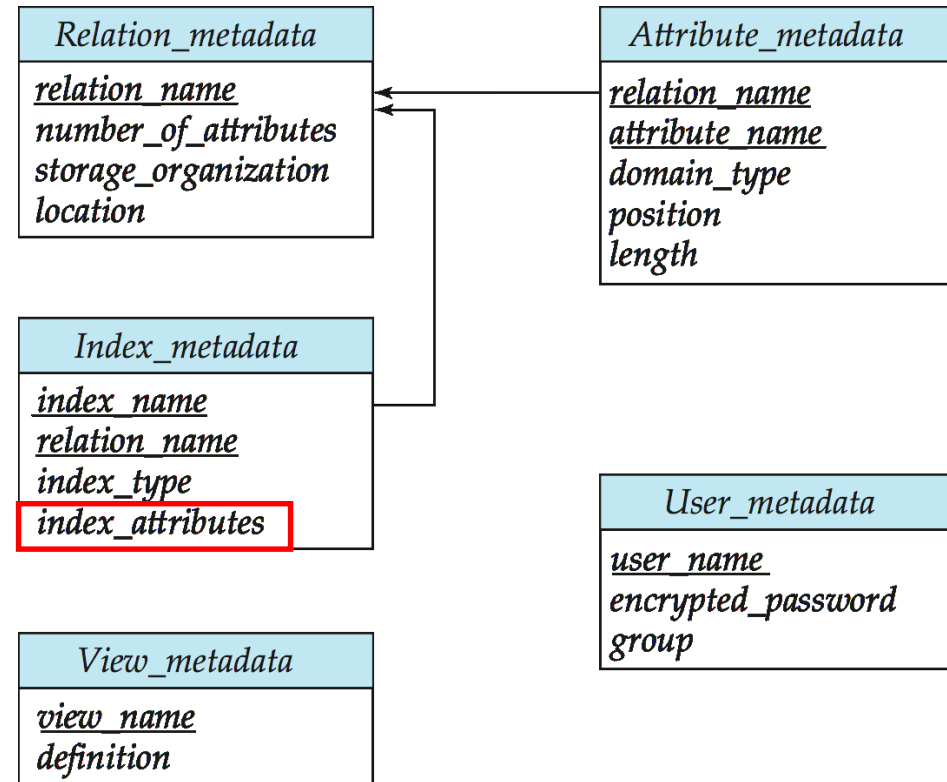
- ▶ All this metadata information constitutes, in effect, *a miniature database*.
- ▶ Generally people prefer to *store the data about the database as relations in the database itself*.
- ▶ Simplify the overall structure of the system and harness the full power of the database for *fast access to system data*.
- ▶ DB system designer have to make choice of *how to represent metadata by relations*.



Relational Representation of System Metadata

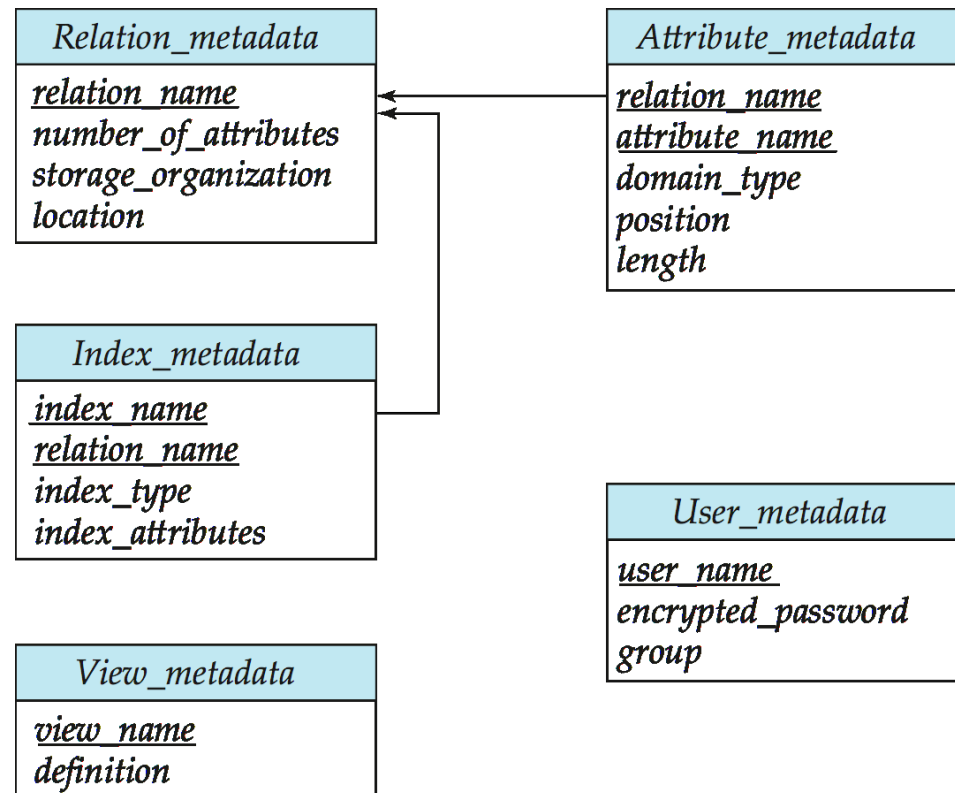
One possible representation:

- The attribute *Index_attributes* of the relation *Index_metadata* is assumed to contain a list of one or more attributes.
- The *Index_metadata* relation is thus not in first normal form.
 - It can be normalized.
 - **But** the above representation is likely to be more efficient to access.
- The data dictionary is often stored in a *nonnormalized* form to achieve *fast access*.



Relational Representation of System Metadata (Cont.)

- ▶ For retrieving records from a relation, DB system will firstly refer to *Relation_metadata* relation to find the location and storage organization of the relation, then fetch records.



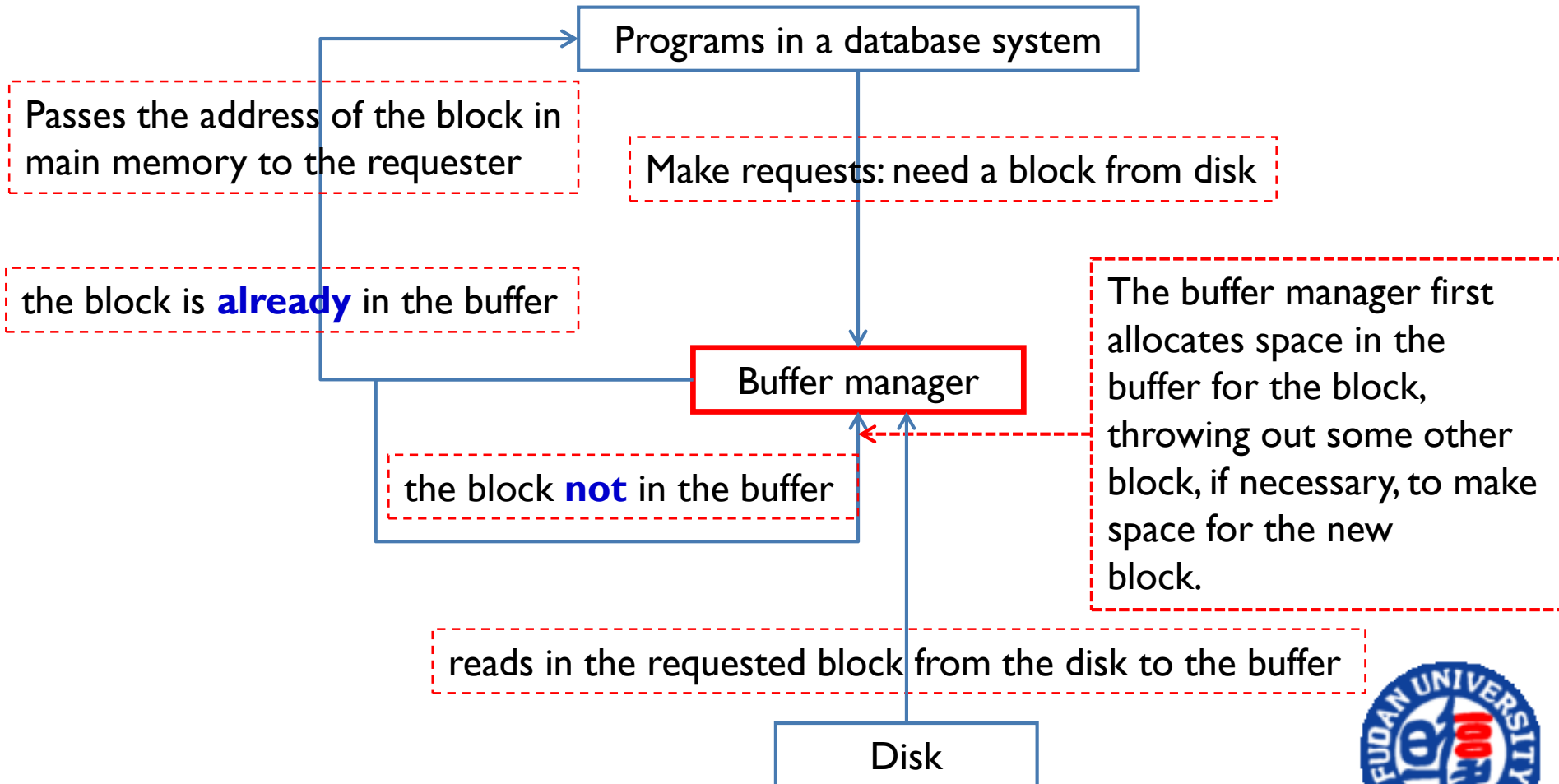
Database Buffer

- ▶ A major goal of the database system is to *minimize the number of block transfers between the disk and memory*.
- ▶ One way to reduce the number of disk accesses is to keep as many blocks as possible in main memory.
- ▶ But, it is not possible to keep all blocks in main memory.
- ▶ We need to manage the allocation of the space available in main memory for the storage of blocks.
- ▶ The **buffer** is *that part of main memory available for storage of copies of disk blocks*.
- ▶ The subsystem responsible for the allocation of buffer space is called the **buffer manager**.



Buffer Manager

► Buffer manager



Techniques used by Buffer Manager

- ▶ The buffer manager appears to be nothing more than a *virtual-memory manager*, like those found in most operating systems.
- ▶ One difference is that the size of the database might be larger than the hardware address space of a machine.
- ▶ Further, to serve the database system well, the buffer manager must use more *sophisticated techniques*:
 - ▶ *Buffer replacement strategy*
 - ▶ *Pinned blocks*
 - ▶ *Forced output of blocks*



Homework

- 10.5 Show the structure of the file of Figure 10.7 after each of the following steps:
- a. Insert (24556, Turnamian, Finance, 98000).
 - b. Delete record 2.
 - c. Insert (34556, Thompson, Music, 67000).
- 10.6 Consider the relations *section* and *takes*. Give an example instance of these two relations, with three sections, each of which has five students. Give a file structure of these relations that uses multitable clustering.



Homework

header				
record 0	10101	Srinivasan	Comp. Sci.	65000
record 1				
record 2	15151	Mozart	Music	40000
record 3	22222	Einstein	Physics	95000
record 4				
record 5	33456	Gold	Physics	87000
record 6				
record 7	58583	Califieri	History	62000
record 8	76543	Singh	Finance	80000
record 9	76766	Crick	Biology	72000
record 10	83821	Brandt	Comp. Sci.	92000
record 11	98345	Kim	Elec. Eng.	80000

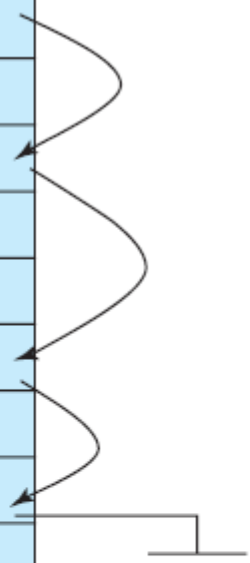


Figure 10.7 File of Figure 10.4, with free list after deletion of records 1, 4, and 6.

